



Instituto Politécnico Nacional

Escuela Superior de Computo



Materia: Aplicaciones para Comunicaciones en Red

Profesor: Tanibet Pérez De los Santos Mondragón

Integrantes: Gil Valentín Oswaldo

Grupo: 6CV1

Fecha: 20/04/2026

Actividad: Ejercicio 3.1 Primitivas de sincronización - Locks

Ejercicio:

Simular el funcionamiento de una cuenta bancaria compartida que es accedida concurrentemente por múltiples hilos. Se debe definir una variable global SALDO que represente el dinero disponible en la cuenta.

SALDO = 10000

Establecemos la variable de SALDO en 10K para poder hacer operaciones con un poco de libertad al momento de manipular el valor de la variable.

Implementar 5 hilos que realicen operaciones de depósito y retiro de dinero sobre dicha cuenta.

```
#''' Creacion y Ejecucion de Hilos '''
threads = []
print(f"=== INICIANDO OPERACIONES BANCARIAS. SALDO INICIAL: ${SALDO} ===\n")

for i in range(5):
    #Enumeramos los hilos
    t = threading.Thread(target=worker, args=(i,), name=f"Cliente-{i+1}")
    threads.append(t)
    t.start()
```

Creamos los 5 hilos y los nombramos y enumeramos para poder identificar cada cliente y que operaciones hacen.

Debido a que varios hilos acceden y modifican el saldo al mismo tiempo, es necesario utilizar mecanismos de sincronización.

```
#####SECCION CRITICA#####
# Solo un hilo a la vez puede entrar a esta estructura `with`
with lock_saldo:
    if operacion == 'deposito':
        logging.debug(f"Iniciando {operacion} de ${monto}...")
        SALDO += monto
        logging.info(f"DEPOSITO EXITOSO. Nuevo Saldo: ${SALDO}")

    elif operacion == 'retiro':
        logging.debug(f"Intentando {operacion} de ${monto}...")
        #Validacion de saldo
        if SALDO >= monto:
            SALDO -= monto
            logging.info(f"RETIRO EXITOSO. Nuevo Saldo: ${SALDO}")
        else:
            logging.warning(f"RETIRO RECHAZADO (Fondos insuficientes). Saldo actual: ${SALDO}")
# Al salir de la estructura `with`, el candado se suelta automaticamente para el siguiente hilo
```

En este caso se usan candados (usando el `lock_saldo`, que en realidad es un candado que está más arriba en el código).

El programa debe asegurar que:

- No se produzcan condiciones de carrera.

```
# Este es el mecanismo de sincronizacion que evitara las condiciones de carrera
lock_saldo = threading.Lock()
```

- El saldo refleje correctamente todas las operaciones realizadas.

```
=== INICIANDO OPERACIONES BANCARIAS. SALDO INICIAL: $10000 ===
[DEBUG] (Cliente-1 ) Intentando retiro de $686...
[INFO] (Cliente-1 ) RETIRO EXITOSO. Nuevo Saldo: $9314
[DEBUG] (Cliente-2 ) Intentando retiro de $1292...
[INFO] (Cliente-2 ) RETIRO EXITOSO. Nuevo Saldo: $8022
[DEBUG] (Cliente-3 ) Iniciando deposito de $630...
[INFO] (Cliente-3 ) DEPOSITO EXITOSO. Nuevo Saldo: $8652
[DEBUG] (Cliente-5 ) Iniciando deposito de $420...
[INFO] (Cliente-5 ) DEPOSITO EXITOSO. Nuevo Saldo: $9072
[DEBUG] (Cliente-4 ) Iniciando deposito de $1723...
[INFO] (Cliente-4 ) DEPOSITO EXITOSO. Nuevo Saldo: $10795
[DEBUG] (Cliente-1 ) Iniciando deposito de $1245...
[INFO] (Cliente-1 ) DEPOSITO EXITOSO. Nuevo Saldo: $12040
[DEBUG] (Cliente-3 ) Intentando retiro de $2688...
[INFO] (Cliente-3 ) RETIRO EXITOSO. Nuevo Saldo: $9352
[DEBUG] (Cliente-2 ) Iniciando deposito de $134...
[INFO] (Cliente-2 ) DEPOSITO EXITOSO. Nuevo Saldo: $9486
[DEBUG] (Cliente-5 ) Iniciando deposito de $906...
[INFO] (Cliente-5 ) DEPOSITO EXITOSO. Nuevo Saldo: $10392
[DEBUG] (Cliente-4 ) Iniciando deposito de $2002...
[INFO] (Cliente-4 ) DEPOSITO EXITOSO. Nuevo Saldo: $12394
[DEBUG] (Cliente-3 ) Iniciando deposito de $422...
[INFO] (Cliente-3 ) DEPOSITO EXITOSO. Nuevo Saldo: $12816
[DEBUG] (Cliente-1 ) Intentando retiro de $2708...
[INFO] (Cliente-1 ) RETIRO EXITOSO. Nuevo Saldo: $10108
[DEBUG] (Cliente-2 ) Intentando retiro de $637...
[INFO] (Cliente-2 ) RETIRO EXITOSO. Nuevo Saldo: $9471
[DEBUG] (Cliente-4 ) Iniciando deposito de $2461...
[INFO] (Cliente-4 ) DEPOSITO EXITOSO. Nuevo Saldo: $11932
[DEBUG] (Cliente-5 ) Iniciando deposito de $2901...
[INFO] (Cliente-5 ) DEPOSITO EXITOSO. Nuevo Saldo: $14833
```

- No se permitan retiros si el saldo es insuficiente.

```
elif operacion == 'retiro':  
    logging.debug(f"Intentando {operacion} de ${monto}...")  
    #Validacion de saldo  
    if SALDO >= monto:  
        SALDO -= monto  
        logging.info(f"RETIRO EXITOSO. Nuevo Saldo: ${SALDO}")  
    else:  
        logging.warning(f"RETIRO RECHAZADO (Fondos insuficientes). Saldo actual: ${SALDO}")
```

Dentro de la sección bloqueante colocamos un if al momento de retirar que permite retirar saldo si es que hay saldo suficiente, si no pues solo se imprime un mensaje que indica que no hay saldo suficiente.

Al finalizar la ejecución de todos los hilos, se debe mostrar el saldo final de la cuenta.

```
=====  
TODOS LOS CLIENTES TERMINARON. SALDO FINAL DE LA CUENTA: $14833  
=====
```